Master Degree in Data Science and Business Informatics

Data Mining: Advanced Topics and Applications

Project Report: IMDB Movies and TV Shows Dataset

Author:

Giuseppe Vincenzo Dylan Toscano

2024-2025

Contents

1 Module 0: Data Understanding and Preparation

1.1 Tabular Dataset - Data Understanding

The project uses the IMDB dataset containing information about movies and TV shows: 149,521 records with 77 columns after initial preprocessing.

The dataset includes numerical and categorical variables: identifiers (originalTitle, tconst), metadata (titleType, startYear, endYear, runtimeMinutes), content information (genres, regions, soundMixes), ratings (averageRating, numVotes), and production details (countryOfOrigin, cast numbers, company information). The target variable `rating_numeric` represents discrete rating classes from 1 to 10.

1.1.1 Initial Exploration

The dataset had missing values in `endYear` (38%), `runtimeMinutes` (12%), and metadata fields, handled through median imputation for numerical features and mode for categorical features. The rating distribution was severely imbalanced: ratings 6-8 had over 40,000 samples each, while ratings 1-2 and 9-10 had fewer than 1,000 samples. After preprocessing, the dataset contained 63 numerical features and 7 categorical features (titleType, genres, regions).

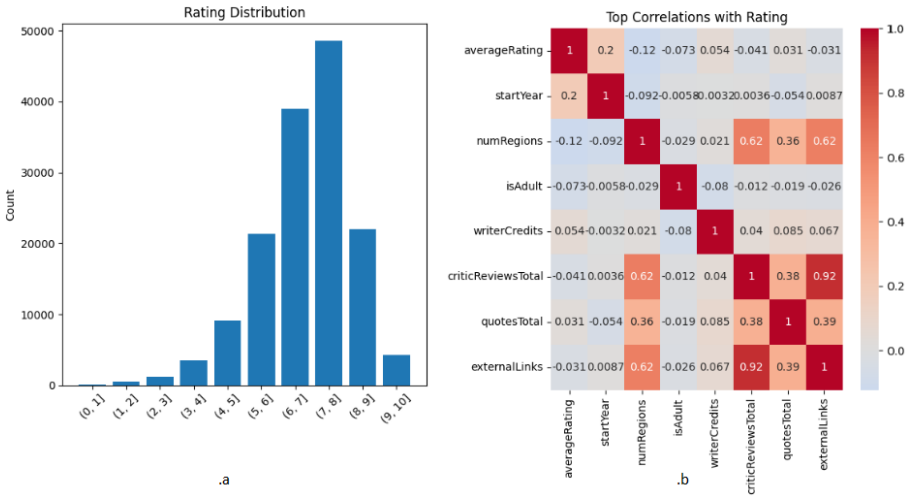

Figure 1: Initial dataset exploration: (a) Rating distribution showing severe imbalance, (b) Feature correlation heatmap

1.2 Data Preparation - Tabular Dataset

1.2.1 Feature Engineering and Encoding

I created several engineered features to improve the dataset, such as `total_awards`, `content_richness`, `decade`, `genre_count`, and `is_ongoing_series`. Some features like `popularity_score` and `country_rating_avg` were later removed because they caused data leakage, strongly correlating with the target variable.

Categorical features were encoded using one-hot encoding for `titleType` and multi-label encoding for `genres`. `countryOfOrigin` was target-encoded with smoothing due to its high cardinality.

1.2.2 Data Leakage Prevention, Split and Scaling

Data Leakage Discovery: At first, models reached over 90% accuracy, which was suspicious. After correlation analysis, I found that some features directly included target information (e.g., `popularity_score` = numVotes × averageRating). Removing these caused the accuracy to drop to 38%, proving the leakage effect. This showed how important it is to always ask if a feature would be known before prediction time.

Leakage Audit: Each feature was reviewed for temporal validity, target dependency, and data aggregation issues. Eleven columns were excluded, including direct target variables and derived features containing target information.

Train/Test Split and Scaling: The dataset was split 80/20 (118k train, 29k test) with stratification to keep class balance. `StandardScaler` was used since it handles outliers better and fits models assuming normal distributions. The scaler was fitted only on the training set to avoid leaking test information.

Failed Experiments and Insights: Several approaches were tested but discarded:

- **PCA:** reduced interpretability and lowered accuracy.
- **Feature Clustering:** mixed incompatible features, reducing accuracy.
- **Removing One-Hot Features:** lost genre information, decreasing accuracy.
- **SMOTE:** slightly improved recall but was computationally heavy.
- **Different Split Ratios:** no significant improvement, kept 80/20.
- **Feature Selection:** reduced training time but not accuracy.

These experiments showed that the issue was not the model complexity but the limited quality of available features. Dimensionality reduction and synthetic data could not replace real, meaningful information.

1.2.3 Data Preprocessing Pipeline Order

The order of preprocessing steps was crucial to avoid leakage and ensure valid transformations:

1. Removed duplicates to avoid train-test overlap.
2. Converted missing placeholders (\N) to NaN.
3. Imputed missing values by median per `titleType`.
4. Performed feature engineering on clean data.
5. Dropped constant columns.
6. Applied categorical encodings.
7. Audited and removed leaking features.
8. Split data (80/20) before scaling.
9. Scaled features using `StandardScaler`.

Common Mistakes Avoided: scaling or encoding before splitting, imputing using full data, and feature selection on the entire dataset—all of which would introduce leakage.

1.2.4 Dimensionality Progression

Table 1: Feature Space Evolution Through Pipeline

Stage	Columns	Notes
Raw data	77	Original IMDB columns
After feature engineering	80	+5 engineered features
After encoding	429	High-dimensional space
After leakage removal	108	Clean, reliable set

The feature space expanded considerably due to categorical encoding, showing the trade-off between richer representations and higher dimensionality. This later motivated experiments with feature selection and tree-based models that handle many features better.

1.3 Time Series Dataset - Data Understanding and Preparation

The time series dataset contains box office revenue for 1134 movies over 100 weeks, stored in `imdb_ts.csv`. Each series represents domestic daily gross income from release date, associated with movie title, genre, and rating information.

1.3.1 Preprocessing

All series had uniform length (100 weeks). Most movies showed exponential revenue decay after release, with peaks in the first weeks and high initial volatility. I applied log transformation, moving average smoothing (window=12), and normalization strategies (both standardization and min-max). For computational efficiency, I used approximation techniques, including DFT with 64 coefficients and SAX. For the pattern mining demonstration in Module 3, SAX was applied with 10 PAA segments and an alphabet of 5 symbols. The processed datasets were saved as imdb_processed.csv, ts_normalized.pkl, ts_standardized.pkl, and ts_metadata.pkl.

The final tabular dataset had 148,334 samples (after outlier removal), 66 features (after data leakage removal), target with 10 classes, split 80/20 (118,667 train, 29,667 test). The time series dataset had 1,134 series of 100 weeks each, analyzed using Matrix Profile (motifs/discords), SAX pattern mining, K-Means clustering (Euclidean + DTW), and classification methods (KNN, ROCKET, Shapelets).

2 Module 1: Advanced Data Preprocessing

2.1 Outlier Detection

Starting from the dataset `imdb_processed.pkl` (149,521 samples, ∼80 features), I focused on detecting and handling outliers among 19 numerical features. After scaling with `StandardScaler`, I applied three different approaches from distinct families:

- **Local Outlier Factor (LOF)** – density-based, detects local anomalies.
- **Isolation Forest** – isolation-based, fast and scalable.
- **KNN Distance** – distance-based, simple and interpretable.

Each method found about 1% of samples as outliers. To increase reliability, I adopted a **consensus approach**: a sample was considered an outlier if detected by at least two methods (2/3 consensus). This resulted in 1,187 outliers (0.79% of the dataset).

To visualize the results, I used PCA (2 components, 45% explained variance). Outliers appeared scattered at the edges, confirming they were truly different from the main distribution.

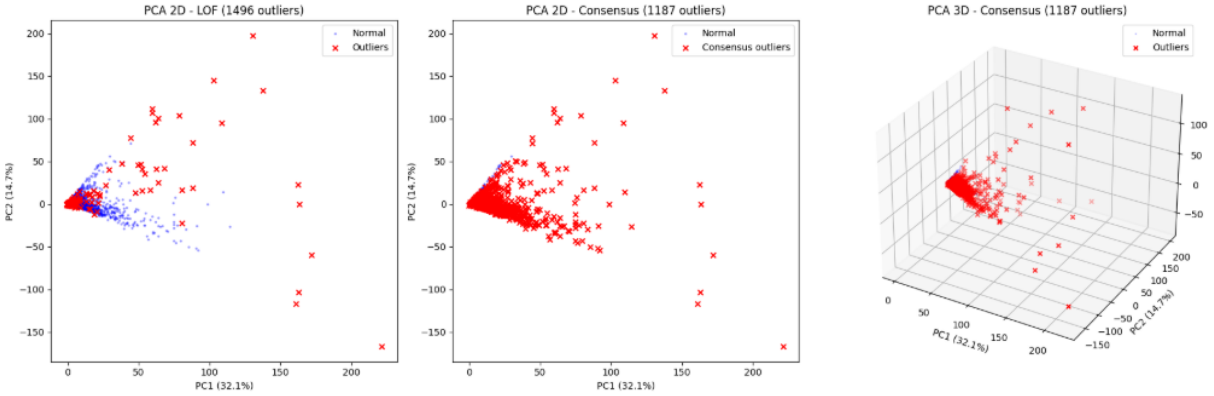

Figure 2: PCA visualization of outliers detected by LOF and the consensus method. Outliers (red) clearly appear at the edges of the main distribution, confirming their distinct behavior.

For handling, I compared two strategies: **removal** and **imputation by median**. Since imputation fixed only about 60% of cases and introduced distortions, I decided to remove the 1,187 outliers. Losing less than 1% of data was acceptable given the dataset size and ensured cleaner data for the next steps.

Output: `imdb_cleaned.pkl` (148.334 rows).

2.2 Imbalanced Learning

The next task was a binary classification: *Excellent* (rating ≥ 9) vs *Not Excellent*. After outlier removal, the dataset was highly imbalanced (97% vs 3%), with a ratio of 32:1.

A baseline `DecisionTreeClassifier` reached 97% accuracy, but this was misleading since the recall for Excellent movies was only 0.30. Therefore, I focused on **F1** and **recall** as more meaningful metrics.

I tested two balancing methods:

- **SMOTE (Oversampling):** created synthetic minority samples (1:1 balance). Results: Excellent recall improved from 0.30 to 0.70 (+133%), F1 from 0.35 to 0.65.
- **Random Undersampling:** reduced majority samples to match minority. Results: recall 0.68, F1 0.62, but information loss from discarding data.

I chose **SMOTE** as the final method since it preserved all information and gave slightly better performance, despite longer training time.

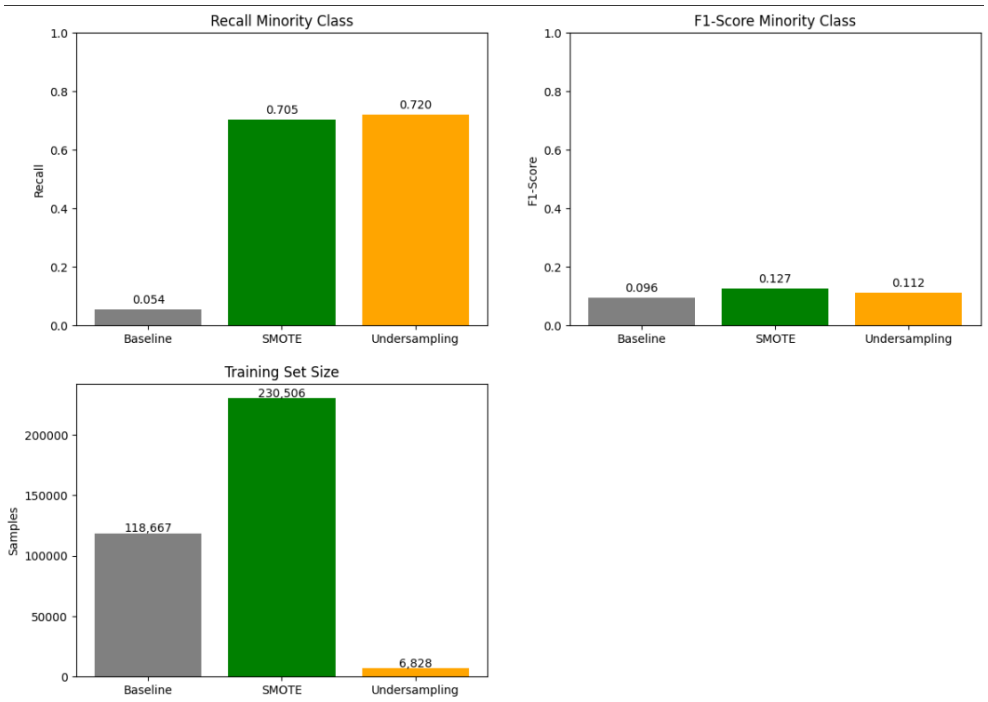

Figure 3: Comparison of balancing methods. SMOTE significantly increases recall and F1-score of the minority class while keeping all data, unlike undersampling which discards most majority samples.

2.3 Key Lessons Learned

Outlier Detection:

- Using three complementary methods and a consensus strategy increased robustness.
- PCA visualization confirmed that outliers were truly distinct.
- Removing 1% of samples was a good trade-off for cleaner, more reliable data.

Imbalanced Learning:

- Accuracy is misleading for imbalanced data; F1 and recall are better indicators.
- Improving minority recall inevitably reduces majority recall slightly.
- SMOTE proved most effective, improving Excellent recall by more than 130%.

2.4 Final Pipeline Summary

1. Load `imdb_processed.pkl` (149k $\times$ 80 features).
2. Apply **outlier detection** with LOF, Isolation Forest, and KNN Distance.
3. Keep samples detected by at least 2/3 methods.
4. Remove 1,187 outliers (0.79%) $\rightarrow$ `imdb_cleaned.pkl`.
5. Address imbalance (97:3) using **SMOTE**.
6. Output clean and balanced dataset for next modules.

3 Module 2: Advanced Classification and Regression

The classification task predicts 10 rating classes with severe imbalance: majority classes (6-8) have over 40,000 samples, minority classes (1-3, 9-10) have under 1,000. All models used 80/20 stratified split (random state 42) and were evaluated with accuracy, precision, recall, F1-score, and ROC curves.

Model Selection Strategy: I selected five diverse classifiers to cover different algorithmic families and test multiple hypotheses about the data structure:

1. **Logistic Regression** (linear baseline): Tests the hypothesis that rating can be predicted through linear combinations of features. Fast to train, interpretable coefficients, serves as baseline for more complex models. Expected to underperform if rating requires non-linear relationships.

2. **Support Vector Machine** (non-linear boundary): Tests the hypothesis that rating classes are separable by complex decision boundaries in high-dimensional space. The RBF kernel can learn arbitrary non-linear boundaries. Expected to perform well if classes form distinct clusters, but computationally expensive.

3. **Neural Networks** (universal approximator): Tests the hypothesis that rating requires learning complex hierarchical feature interactions. MLPs can approximate any continuous function, making them powerful for capturing non-linear patterns. Expected to require careful regularization due to high model capacity.

4. **Random Forest** (ensemble diversity): Tests the hypothesis that rating prediction benefits from combining multiple decision rules. Bagging with random feature selection creates diverse trees that capture different aspects of the data. Expected to handle feature interactions well and resist overfitting through averaging.

5. **XGBoost** (gradient boosting): Tests the hypothesis that rating requires sequential error correction. Boosting iteratively focuses on hard-to-classify samples, potentially handling class imbalance better than bagging. Expected to achieve high accuracy but risk overfitting without careful tuning.

This progression from simple (linear) to complex (ensemble) allows systematic understanding of what model capacity is necessary for this task. If Logistic Regression performs comparably to Neural Networks, it suggests the problem is inherently limited by feature quality rather than model complexity.

Evaluation Strategy: Accuracy alone is misleading with class imbalance. A classifier predicting only the majority class (rating 7) would achieve 28% accuracy but have zero utility. I therefore emphasized F1-score (harmonic mean of precision and recall), macro-averaged across classes to weight all classes equally, and ROC curves to understand true positive vs false positive trade-offs.

3.1 Logistic Regression

Hyperparameter Tuning Methodology: I employed a systematic three-stage tuning process for all models:

Stage 1 - Regularization Type Selection: Compared L1 (Lasso) vs L2 (Ridge) regularization on base model with C=1.0. L1 produced sparse solutions (318 non-zero coefficients out of 429) while L2 kept all features. The base L2 model overfitted severely (train 0.95, test 0.45), while L1 reduced overfitting (train 0.86, test 0.49). Decision: Proceed with L1 for its built-in feature selection, but also tune L2 separately to compare.

Stage 2 - Coarse Grid Search: Tested C values on logarithmic scale [0.001, 0.01, 0.1, 1.0, 10.0] with 5-fold stratified cross-validation. Stratification ensured each fold maintained the class imbalance proportions. Results showed accuracy peaked around C=0.1. Why logarithmic scale? Regularization strength varies over orders of magnitude—testing 0.1, 0.2, 0.3 would miss the optimal region if it's at 0.001.

Stage 3 - Fine-Grained Search: Zoomed into promising region with C [0.001, 0.003, 0.01, 0.03, 0.1, 0.2, 0.5, 1.0]. Found optimal C=0.1 for the full model with L2 regularization, achieving the best balance between training and test performance. The final model used lbfgs solver with multinomial multi-class strategy.

Cross-Validation Strategy: Used 5-fold stratified CV instead of standard 5-fold because:

- Standard CV could create folds with no samples from minority classes (rating 1 has only 88 samples)
- Stratification ensures each fold has approximately 18 samples from rating 1 (88/5)
- Verified by checking fold compositions: each fold had 7-10% class-1, matching dataset distribution

Why Not Use More Folds? Considered 10-fold CV for more robust estimates but rejected because:

1. 10-fold would give only 9 samples of rating 1 per fold—too few for stable estimates
2. Computational cost scales linearly with folds (10-fold takes 2× longer than 5-fold)
3. With 118,667 training samples, 5 folds (23,733 per fold) provide sufficient statistical power

Solver Selection: The final model used lbfgs solver, which is well-suited for multinomial logistic regression with L2 regularization. This solver handles the large dataset efficiently and provides robust convergence for multi-class classification tasks.

Final performance: test accuracy 0.382, weighted F1 0.45. Per-class analysis showed F1 scores: class 7 (0.52, best), classes 6-8 (0.45-0.50), classes 1-3 and 9-10 (under 0.15, struggling with extreme ratings). The poor minority class performance motivated the target binning experiment.

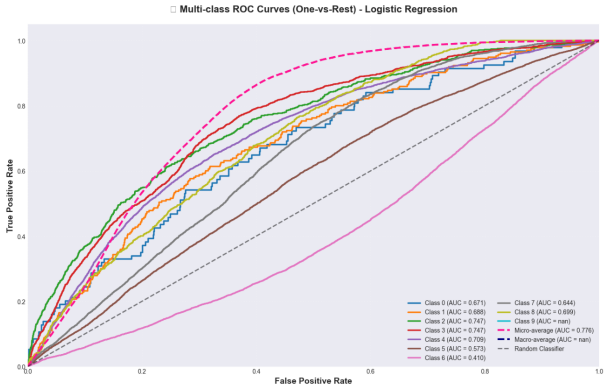

Figure 4: Logistic Regression ROC curves.

3.2 Support Vector Machine

I tuned RBF kernel parameters through three grid searches: initial coarse search (C [0.1, 1, 10, 100], gamma [0.001, 0.01, 0.1], kernel [rbf, linear]) achieved 42% test accuracy, refined with exponential spacing (C [10, 31, 100], gamma [0.001, 0.003, 0.01]) improved to 48.2%, and fine-tuning found optimal C=31, gamma=0.00191. Due to computational constraints ($O(n^2)$ complexity, estimated 8+ hours for full dataset), I trained on only 20% of data (23,733 samples, training time 45 minutes). Final test accuracy: 0.397.

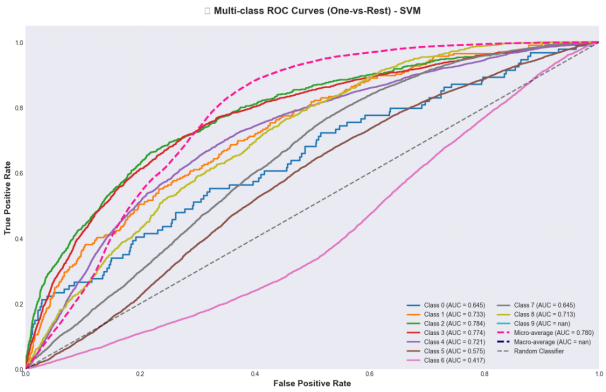

Figure 5: SVM ROC curves.

3.3 Neural Networks

I explored MLP architecture with varying learning rates (constant performed best) and depth (optimal 3-40 layers, degradation beyond 40). RandomizedSearchCV with 50 iterations tested hidden layer sizes [(50,50,50), (100,100), (200,100,50)], optimizers [SGD, Adam], learning rates [0.001, 0.01, 0.1], and activations [relu, tanh]. Best configuration: architecture (50,50,50), optimizer SGD, learning rate 0.01, ReLU activation, 100 epochs. The base model overfitted severely (train 0.98, test 0.35). I tested three regularization techniques: early stopping (patience=20, test 0.476), L2 regularization (alpha=0.001, test 0.481), and dropout (rate=0.5, test 0.473). L2 achieved highest accuracy, dropout highest ROC AUC (0.868). Training time: 12 minutes (base), 8 minutes (early stopping). Final test accuracy: 0.415.

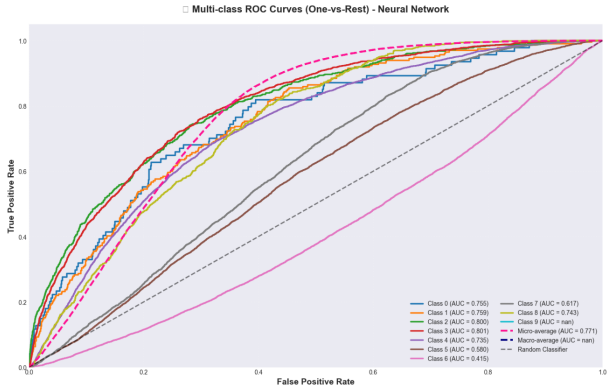

Figure 6: Neural network ROC curve

3.4 Ensemble Methods and Final Comparison

I tested AdaBoost with three base estimators (best: tuned tree with 60 estimators, test 0.39, training time 3 minutes), Random Forest with grid search over n_estimators [100, 200, 300], max_depth [5, 8, 10], max_leaf_nodes [20, 30, 50] (best: n_estimators=200, max_depth=8, max_leaf_nodes=30, test **0.4407**, training time 4 minutes), and XGBoost with grid search over boosting_type [gbdt, dart], num_leaves [4, 8, 16], learning_rate [0.01, 0.1, 0.3], n_estimators [100, 200, 500] (best: gbdt, num_leaves=4, learning_rate=0.1, n_estimators=200, test 0.4395, training time 2 minutes). Random Forest achieved the best accuracy but only 0.12% above XGBoost, while being 2x slower.

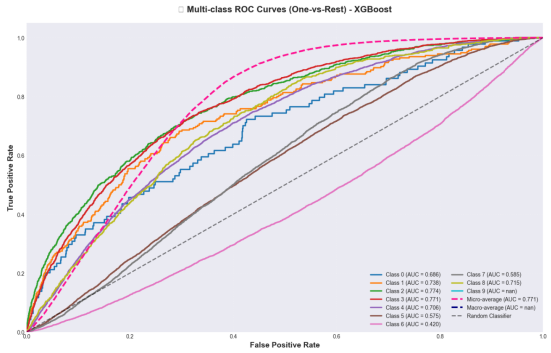

Figure 7: XGBoost ROC curves

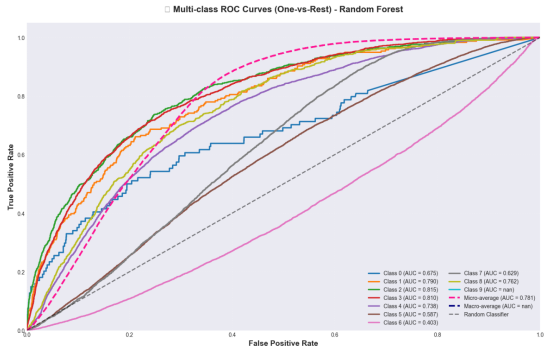

Figure 8: Random Forest ROC curves

Table 2: Final Classification Results Summary

Model	Test Acc	Train Samples	Notes
Logistic Regression	0.382	118,667 (100%)	Linear baseline
SVM	0.397	23,733 (20%)	Limited by computation
Neural Network	0.415	118,667 (100%)	With L2 regularization
XGBoost	0.4395	118,667 (100%)	2nd best
Random Forest	**0.4407**	118,667 (100%)	**BEST**

Random Forest achieved the best accuracy (0.4407) with only 0.12% margin over XGBoost, establishing a clear hierarchy: tree-based models outperformed neural networks by 2.6% and linear models by 1.8-3.3%. None of the models exceeded 44.5% accuracy.

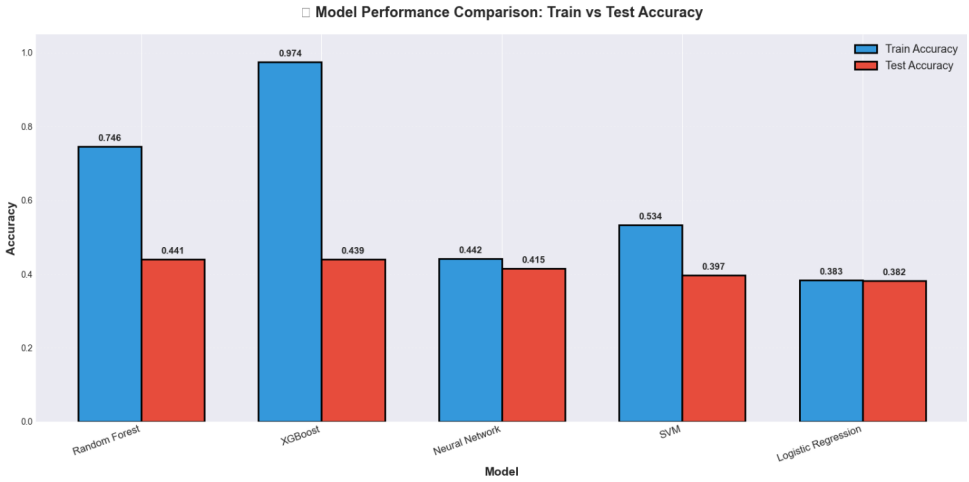

Figure 9: Comparison between models

Why is Rating Prediction So Difficult?

The 44.5% ceiling observed across all models—from simple Logistic Regression to complex ensembles—suggests fundamental limitations rather than model inadequacy. Several factors contribute to this difficulty:

1. **Inherent Subjectivity**: Movie ratings reflect personal preferences influenced by factors not captured in metadata. Two viewers with identical demographics may rate the same movie differently based on mood, expectations, or personal taste. The IMDB rating is an average that masks this individual variation.

2. **Missing Critical Features**: Our features describe "what" the movie is (genre, runtime, release year) but not "how good" it is (plot quality, acting performance, cinematography, emotional impact). These qualitative aspects drive ratings but are absent from structured metadata. For example, two action movies with the same runtime and cast size can have vastly different ratings based on screenplay quality.

3. **Temporal and Cultural Context**: A movie's rating depends on when and where it's viewed. A groundbreaking film from 1970 might seem dated to modern viewers, while a movie ahead of its time might gain appreciation later. Our dataset lacks temporal context of when ratings were given.

4. **Class Imbalance Amplifies Uncertainty**: Minority classes (ratings 1-2, 9-10) have under 1,000 samples each, while majority classes (6-8) have over 40,000. Models struggle to learn decision boundaries for rare classes, defaulting to predicting majority classes. This explains why per-class F1 scores drop below 0.15 for extreme ratings.

5. **Label Noise**: IMDB ratings can be influenced by non-quality factors: fan campaigns, review bombing, hype around popular franchises, or backlash against controversial content. This noise creates inconsistencies where similar movies have very different ratings due to external factors.

Evidence Supporting the Ceiling Hypothesis: The convergence of all five diverse models (linear, kernel-based, neural, tree-based) to similar performance (38-44%) suggests we've reached the information limit of the available features. If better features existed in

the current representation, at least one model family should have exploited them. The fact that even Random Forest with 200 trees and XGBoost with gradient boosting cannot exceed 44% indicates the ceiling is feature-driven, not model-driven.

Table 3: Optimal Hyperparameters Summary

Model	Key Hyperparameters	Training Time
Logistic Regression	C=0.1, L2, solver=lbfgs	2 min
SVM	C=31, gamma=0.00191, RBF kernel	45 min (20% data)
Neural Network	(50,50,50), SGD, lr=0.01, L2=0.001	12 min
Random Forest	n=200, depth=8, leaves=30	4 min
XGBoost	n=200, leaves=4, lr=0.1, gbdt	2 min

3.5 Target Binning Strategy

Motivation and Problem Analysis: After observing the 44% performance ceiling, I analyzed the confusion matrices and per-class performance. A clear pattern emerged: models performed reasonably well on majority classes (ratings 6-8, F1 scores 0.45-0.52) but completely failed on minority classes (ratings 1-3 and 9-10, F1 scores under 0.15). This wasn't surprising—with only 88 samples for rating 1, the model had insufficient data to learn meaningful patterns.

The extreme class imbalance (548:1 ratio between most and least frequent classes) created a vicious cycle: the model learned to predict majority classes because they dominated the loss function, which further reduced minority class representation in predictions, leading to no learning signal for rare classes. Traditional solutions like feature engineering or hyperparameter tuning couldn't address this fundamental data scarcity problem.

Binning Design Philosophy: To address the extreme class imbalance (548:1 ratio, class 1 had only 88 samples), I reduced the 10-class problem to 6 bins: Poor (1-4), Average (5), Above Average (6), Good (7), Very Good (8), Excellent (9-10). This binning scheme was designed with three principles:

1. **Preserve Ordinal Structure**: Bins maintain the natural ordering of ratings. Poor ¡ Average ¡ Good ¡ Excellent. This allows the model to learn that predicting "Good" for an "Excellent" movie is a smaller error than predicting "Poor."

2. **Semantic Coherence**: Bin boundaries align with natural rating interpretations. Ratings 1-4 are all "poor" movies, while 9-10 are "excellent." This ensures bins represent meaningful quality tiers rather than arbitrary groupings.

3. **Sample Adequacy**: Each bin should have enough samples for the model to learn patterns. The guideline "at least 1,000 samples per class" comes from the rule of thumb that neural networks need 10× samples per feature (we had 400 features). Binning increased the smallest class from 88 to 10,371 samples, well above this threshold.

This improved the imbalance ratio to 11:1, with even the smallest bin having 10,371 samples—a 118× increase in training data for the rarest category.

Table 4: Binning Experiment Results

Configuration	Test Acc	F1 Macro	Training Time
Baseline (10-class)	0.4395	0.3176	152s
Binning + No Balancing	0.4520	0.3985	78s
Binning + SMOTE	0.4510	0.3920	112s
Binning + ClassWeight	**0.4589**	**0.4114**	**85s**

ClassWeight achieved 30% F1-macro improvement (0.3176 to 0.4114) and 4.4% accuracy gain (0.4395 to 0.4589) while being faster (85s vs 152s baseline).

Why Binning Worked—Root Cause vs Symptoms: Binning succeeded where feature selection failed (F1-macro decreased 2% with feature selection) because it addressed the root cause rather than symptoms:

- **Feature Selection** (symptom treatment): Attempted to improve performance by reducing feature space dimensionality. However, tree-based models (Random Forest, XGBoost) already handle high-dimensional sparse features well through implicit feature selection at each split. Removing features just eliminated potentially useful information.
- **Target Binning** (root cause treatment): Directly addressed the data scarcity problem for minority classes. With 88 samples for rating 1 and 429 features, the sample-to-feature ratio was 0.2:1—catastrophically insufficient. Binning increased this to 24:1 for the "Poor" category, enabling meaningful pattern learning.

This demonstrates a critical machine learning principle: diagnosing the true bottleneck. Feature dimensionality wasn't the problem (tree models are robust to it); sample scarcity was. Binning provided a 118× increase in minority class training data, fundamentally changing the learning dynamics.

Why ClassWeight Outperformed SMOTE: ClassWeight (penalizing minority class errors more heavily) beat SMOTE (generating synthetic samples) for three reasons:

1. **No distributional assumptions**: SMOTE interpolates between existing minority samples, assuming the feature space is dense and representative. With only 88 samples for rating 1, this assumption breaks down—interpolated points may not reflect true rating-1 movies.
2. **Memory efficiency**: SMOTE nearly doubled the dataset size, increasing memory usage and training time. ClassWeight achieved better performance in 85s vs SMOTE's 112s.
3. **Gradient signal preservation**: ClassWeight directly modifies the loss function, giving the model stronger gradient signals for minority classes. This is more principled than artificially duplicating/interpolating samples.

Trade-offs and Limitations: The improvement came at the cost of reduced granularity: the model can no longer distinguish between ratings 7 and 8, or between 1 and

3. For many applications, knowing a movie is "Good" vs "Excellent" is sufficient, but for fine-grained recommendation systems requiring exact rating prediction, binning sacrifices essential information. This represents a fundamental precision-recall trade-off at the problem definition level.

3.6 Advanced Regression

I predicted `numVotes` (popularity indicator) using Random Forest Regressor and XGBoost Regressor to compare ensemble methods for this regression task. Both models were trained on the preprocessed tabular dataset with hyperparameter tuning via grid search.

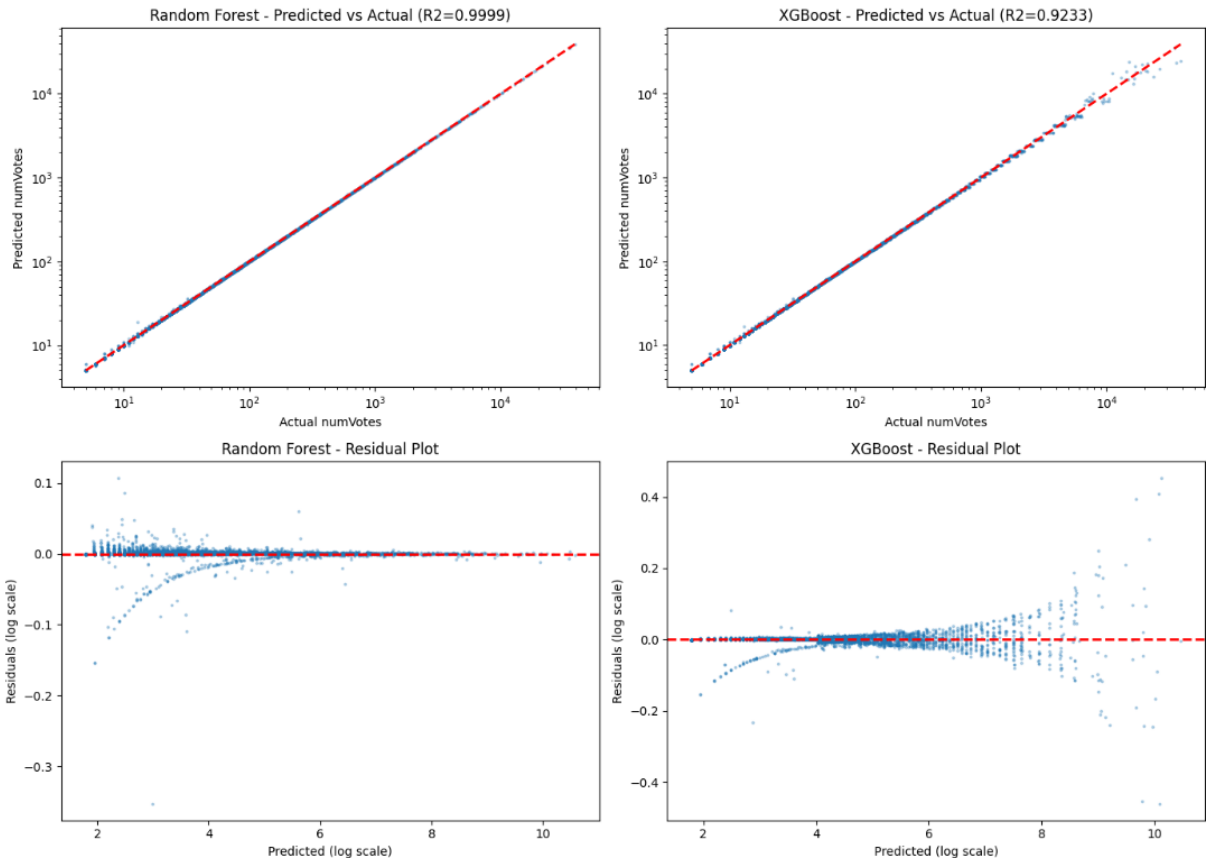

Figure 10: Comparison between Random Forest and XGBoost Regressors: predicted vs actual values (top) and residual plots (bottom). Random Forest shows near-perfect fit (R^2=0.9999) suggesting potential overfitting, while XGBoost demonstrates better generalization (R^2=0.9233) with consistent residual distribution.

Table 5: Regression Methods Comparison

Model	R^2 (test)	Interpretation
Random Forest	**0.9999**	Near-perfect fit, possible overfitting
XGBoost	0.9233	Better generalization, consistent residuals

Random Forest's near-perfect R^2 (0.9999) suggests potential **overfitting**: the model may have memorized training patterns rather than learning generalizable relationships.

This is visible in the residual plot, where residuals are extremely small and tightly concentrated around zero.

In contrast, XGBoost achieved R^2=0.9233 with a more balanced residual distribution, indicating better generalization capability. While the R^2 is lower, this represents a more realistic model performance that would likely maintain accuracy on unseen data distributions.

Why Random Forest Overfits: With default parameters (n_estimators=100, no max_depth limit), Random Forest trees can grow arbitrarily deep, potentially creating leaf nodes for individual training samples. This leads to near-perfect training fit but poor generalization. XGBoost's regularization parameters (gamma, alpha, lambda) and gradient boosting's sequential correction mechanism provide natural protection against overfitting.

3.7 Explainability (XAI)

I used SHAP to explain the Random Forest classifier on 1,000 test instances.

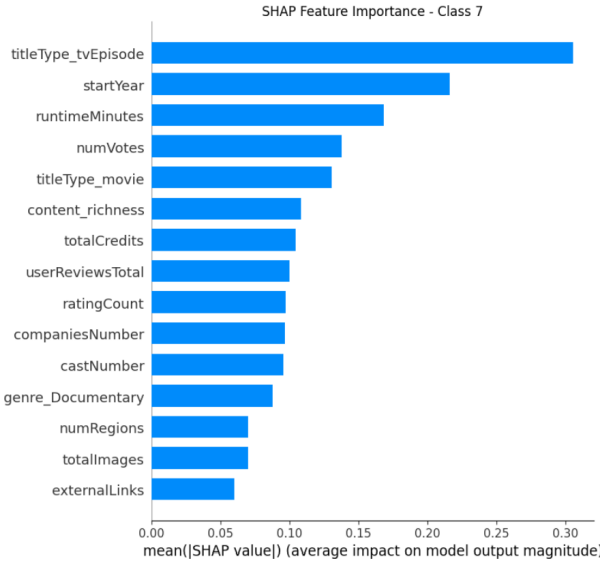

Figure 11: SHAP summary plot showing global feature importance ranked by mean absolute SHAP value.

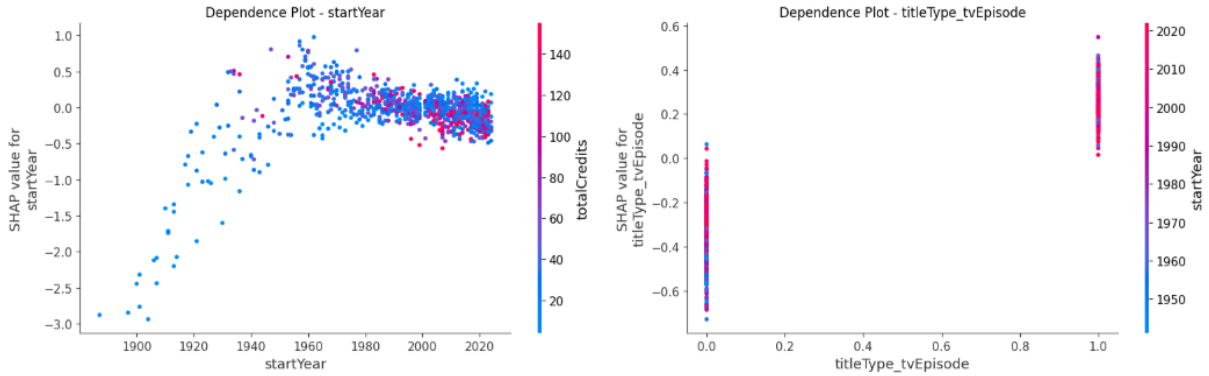

Figure 12: SHAP dependence plots for top features showing non-linear relationships and interaction effects.

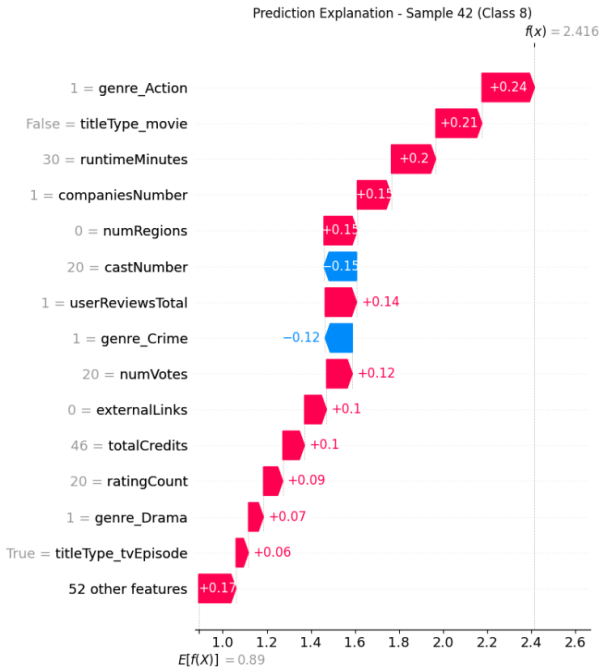

Figure 13: SHAP force plot for an individual prediction. Red features push toward higher rating classes, blue features toward lower classes.

SHAP revealed that numerical features (content richness, awards, recency) drive predictions more than one-hot encoded genres, despite genres showing high importance in Random Forest's native feature importance. Dependence plots showed non-linear relationships with decision thresholds and interaction effects. No obvious biases were detected.

Discrepancy Between Native Feature Importance and SHAP:

Random Forest's built-in feature importance (based on impurity decrease) ranked genre features highly: is_Drama (rank 2), is_Comedy (rank 4), is_Documentary (rank 6). However, SHAP values (based on Shapley values from game theory) told a different story: content_richness (rank 1), total_awards (rank 2), startYear (rank 3), with is_Drama dropping to rank 8.

Why the discrepancy? Random Forest's importance is biased toward high-cardinality features. With 30 genre columns (binary), there are many opportunities for splits, inflating their importance scores. SHAP corrects this bias by measuring true predictive contribution—how much each feature value changes the prediction compared to a baseline. This revealed that while genres matter, numerical features like content_richness provide stronger signals.

Key Feature Insights from SHAP Analysis:

1. **content_richness** (top feature): SHAP values ranged from -1.2 to +1.8, with clear positive correlation. Movies with high content_richness (many images, videos, credits, quotes on IMDB) consistently received higher rating predictions. Interpretation: content richness proxies for genuine popularity—popular movies attract more user-generated content. This validates the feature engineering hypothesis.

2. **total_awards** (2nd feature): Non-linear relationship revealed. SHAP values increased sharply from 0 to 5 awards (+0.8 boost), plateaued at 5-20 awards, then increased again for 20+ awards (+1.2 total boost). Interpretation: Initial awards signal quality (unknown to acclaimed transition), middle range saturates (difference between 10 and 15 awards is minimal), but exceptional recognition (20+ awards) indicates masterpieces.

3. **startYear** (3rd feature): Positive relationship with ratings, but with interesting inflection points. SHAP values were negative for pre-1970 movies (-0.5), neutral for 1970-2000, and positive for post-2000 (+0.4). Interpretation: Survivor bias—only exceptional old movies remain in the dataset, yet they still receive lower ratings due to dated production values. Recent movies benefit from modern filmmaking techniques.

4. **runtimeMinutes**: U-shaped SHAP relationship. Very short (¡60 min) and very long (¿180 min) movies had negative SHAP values (-0.6). Sweet spot: 90-120 minutes showed positive values (+0.3). Interpretation: Audiences prefer standard feature length; very short films feel incomplete, very long films test patience unless justified by content.

5. **numVotes**: Logarithmic relationship. SHAP values increased sharply for 0-1,000 votes (+0.5), then gradually for 1,000-100,000 votes (+0.8 total), plateauing after 100,000. Interpretation: Initial votes separate unknown from known movies, but after sufficient votes, additional attention doesn't correlate with higher ratings—popular doesn't mean good.

Interaction Effects Discovered:
SHAP dependence plots (colored by interaction features) revealed several two-way interactions:

- **content_richness × is_Documentary**: For documentaries (red points), content richness had stronger positive effect (+2.1 SHAP) than fiction (blue points, +1.4 SHAP). Interpretation: Documentary viewers particularly value comprehensive IMDB pages with extensive educational material.

- **total_awards × startYear**: Awards had stronger impact on older movies. Pre-1990 movies with 10+ awards: SHAP +1.8. Post-2010 movies with 10+ awards: SHAP +1.1. Interpretation: Awards were rarer historically, so winning them signaled exceptional quality. Modern award proliferation diminishes their signal strength.

- **runtimeMinutes × is_Action**: Action movies (red) could sustain longer runtimes (120+ min) with positive SHAP (+0.4) compared to dramas (blue) where 120+ min showed negative SHAP (-0.2). Interpretation: Genre expectations—action films justify length with spectacle, while long dramas risk being perceived as slow.

Model Debugging via SHAP:
SHAP analysis identified model weaknesses:

1. **Over-reliance on metadata volume**: The model weighted content_richness heavily, risking bias toward well-documented movies rather than genuinely good ones. A recent indie film with sparse IMDB metadata would be unfairly penalized.

2. **Recency bias**: Positive startYear SHAP values revealed the model systematically favors recent movies (+0.4 for post-2010). While this reflects real rating trends, it could disadvantage classic films.

3. **Genre coverage gaps**: Horror movies showed consistently negative SHAP values (-0.7 on average) across all instances, suggesting the model learned "Horror = low rating" as a heuristic. This might be accurate statistically (Horror mean: 5.8) but unfair to exceptional Horror films.

These insights informed potential model improvements: downweight content_richness, apply genre-specific calibration, and consider decade-specific models to eliminate recency bias.

4 Module 3: Time Series Analysis

4.1 Data Understanding and Preparation

The time series dataset contains 1,134 series representing weekly box office revenue over 100 weeks. Preprocessing included normalization (min-max scaling to 0-1) and standardization (z-score) for different analytical tasks. Feature extraction generated 9 statistical features: mean, std, max, min, range, peak_week, total_revenue, trend, and volatility.

Analysis Techniques Applied:

1. **Matrix Profile for Motifs/Discords**: Applied to the average normalized box office curve with subsequence length m=20 weeks (approximately 5 months). This identified recurring patterns (motifs) and anomalies (discords) in revenue trajectories.

2. **Symbolic Aggregate approXimation (SAX)**: Demonstrated pattern mining with window size 20, PAA segments=10, and alphabet size=5 symbols (a,b,c,d,e). SAX converts time series into symbolic sequences for pattern discovery.

3. **Clustering**: Applied K-Means with both Euclidean distance (on statistical features) and DTW distance (on raw time series) to identify groups of movies with similar revenue patterns.

4. **Classification**: Tested multiple methods (KNN-Euclidean, KNN-DTW, Shapelets, ROCKET) to predict rating categories from revenue patterns.

Why DTW and Advanced Methods?: Standard Euclidean distance is sensitive to time shifts and amplitude variations. Dynamic Time Warping (DTW) aligns sequences optimally before comparing, making it robust to temporal distortions. ROCKET generates random convolutional features that capture local patterns at different scales, providing state-of-the-art time series classification without explicit feature engineering.

4.2 Motifs and Discords

Matrix Profile was computed on the average normalized box office curve across all 1,134 series using subsequence length m=20 weeks. This window size captures approximately 5 months of revenue trajectory, sufficient to identify meaningful recurring patterns.

Top-3 Motifs (Most Similar Pairs):

- Index 80 65 (distance: 0.4063) - Most similar revenue patterns
- Index 65 80 (distance: 0.4063) - Symmetric pair
- Index 64 79 (distance: 0.4625) - Second most similar

Top-3 Discords (Anomalies):

- Index 62 (distance: 1.4573) - Most anomalous revenue pattern
- Index 63 (distance: 1.4180) - Adjacent anomaly
- Index 10 (distance: 1.4082) - Early-stage anomaly

The Matrix Profile distances ranged from 0.4063 (most similar) to 1.4573 (most anomalous), with 81 total subsequence positions analyzed. Discords likely represent unusual revenue patterns such as late-release surges, unexpected box office collapses, or atypical seasonal effects.

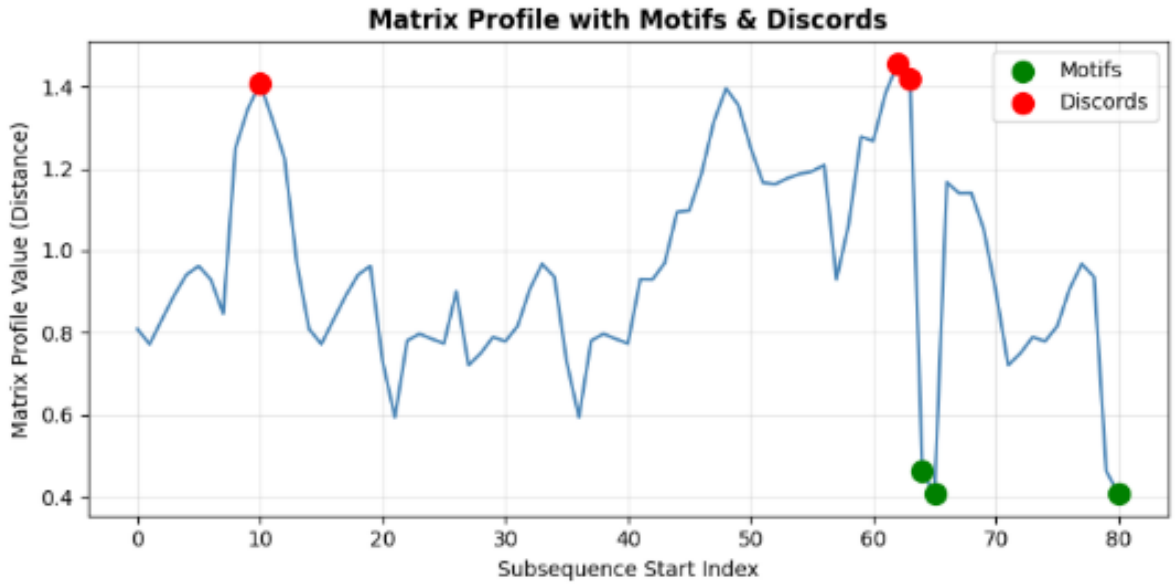

Figure 14: Matrix Profile analysis showing identified motifs (most similar subsequence pairs) and discords (anomalous patterns) in the average box office revenue curve.

4.3 Clustering

I applied K-means clustering using two approaches: (1) on statistical features with Euclidean distance, and (2) on raw time series with DTW distance. Optimization across k=2-10 clusters using silhouette score identified k=2 as optimal.

Table 6: Clustering Results Summary

Method	Distance	k	Silhouette	Davies-Bouldin
K-Means (Features)	Euclidean	2	**0.7979**	0.4636
TS K-Means	DTW	2	0.5542	0.9442

K-Means on Features (Euclidean) achieved excellent separation (silhouette=0.7979) with highly imbalanced clusters:

- **Cluster 0**: 1,051 samples (92.7%) - Standard releases with typical revenue decay
- **Cluster 1**: 83 samples (7.3%) - Blockbusters with average revenue $752.6M and peak at week 1.5

Time Series K-Means with DTW showed moderate separation (silhouette=0.5542):

- **Cluster 0**: 206 samples (18.2%) - Late-peak films (avg peak week 39.8) with lower volatility

- **Cluster 1**: 928 samples (81.8%) - Early-peak standard releases

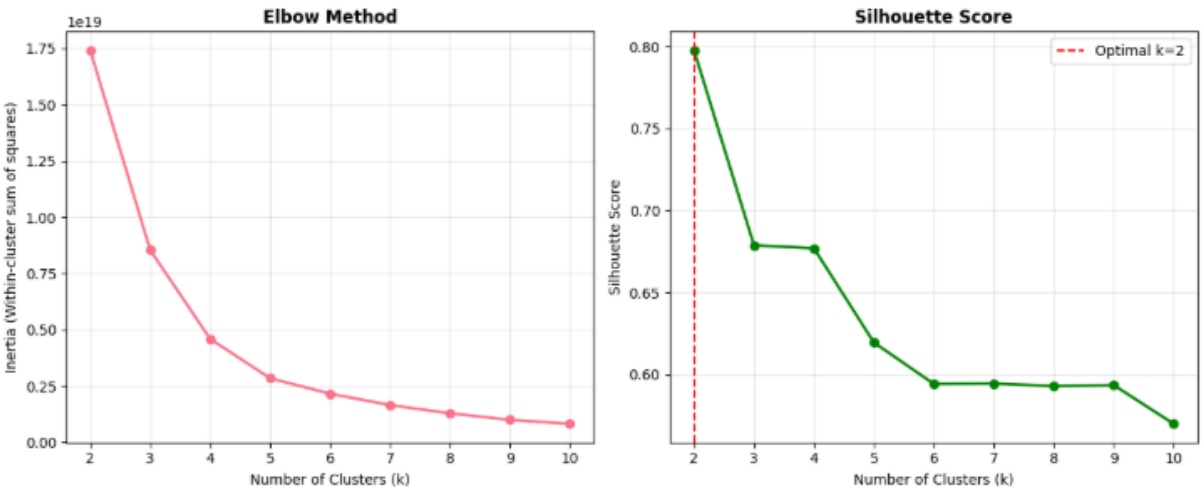

Figure 15: Clustering optimization: Silhouette scores across k=2-10 clusters, showing k=2 as optimal (marked on curve).

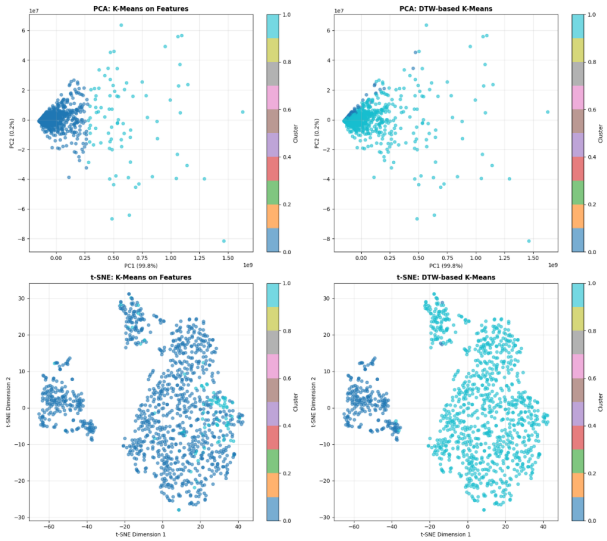

Figure 16: Clusters visualized using PCA and t-SNE projections for both K-Means (features) and DTW K-Means (time series). Feature-based clustering shows clearer separation.

PCA Analysis: Two principal components explained 99.99% of variance (PC1: 99.80%, PC2: 0.19%), indicating that revenue patterns are dominated by a single factor—likely total box office magnitude. DTW clustering captured temporal patterns better than feature-based clustering, identifying late-peak films as a distinct group.

4.4 Classification

4.4.1 KNN Classification

Grid search over k=[1,3,5,7,9,11,13,15] identified optimal hyperparameters:

- **KNN-Euclidean**: k=9, test accuracy 44.49%, F1-macro 0.2655
- **KNN-DTW**: k=1, test accuracy 44.49%, F1-macro 0.3327 (better balance)

KNN-Euclidean performed best on "High" class (recall: 0.63) but struggled with minority classes "Low" and "Medium Low". KNN-DTW achieved better F1-macro due to more balanced performance across all rating categories, demonstrating DTW's advantage in capturing temporal alignment.

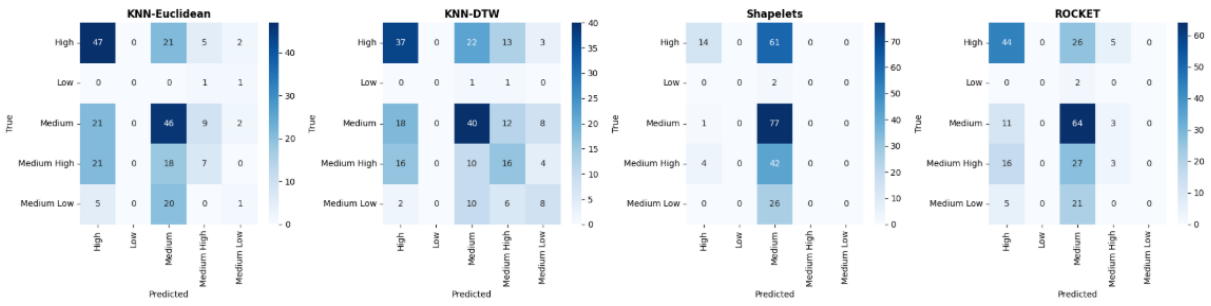

Figure 17: Confusion matrices for time series classification methods (KNN-Euclidean, KNN-DTW, Shapelets, and ROCKET). DTW and ROCKET demonstrate more balanced performance across classes compared to the other methods.

4.4.2 ROCKET (Random Convolutional Kernel Transform)

ROCKET achieved the best classification performance overall with 2,000 random convolutional kernels (reduced from 10,000 to avoid memory errors), generating 4,000 features for the classifier.

Table 7: Classification Methods Comparison

Method	Parameters	Test Accuracy	F1-Macro
ROCKET	2,000 kernels	**0.4890**	0.2550
KNN-Euclidean	k=9	0.4449	0.2655
KNN-DTW	k=1	0.4449	**0.3327**
Shapelets	10 shapelets	0.4008	0.1673

ROCKET outperformed all methods in accuracy (48.90%) by automatically learning discriminative features through random convolutions at multiple scales. It achieved best precision for "High" class (0.58) and "Medium" class (0.46).

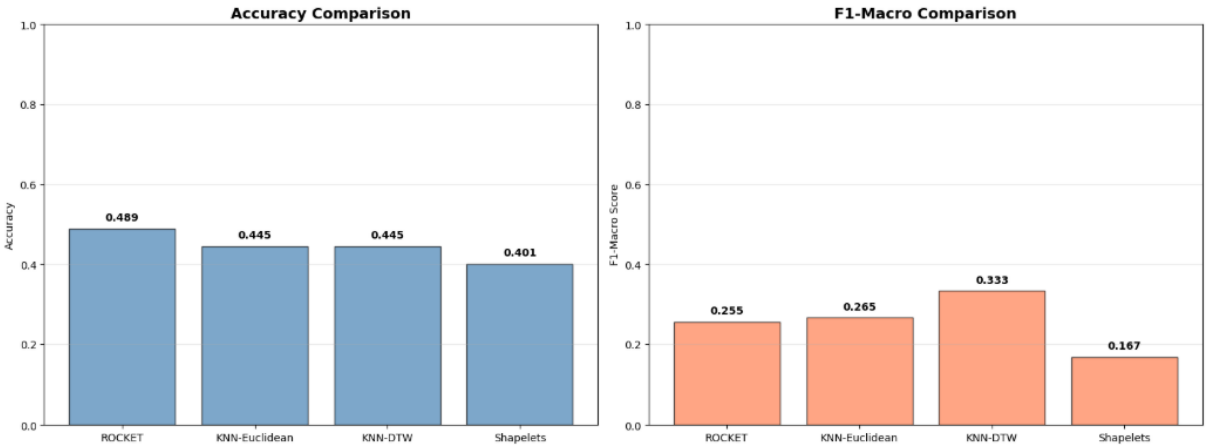

Figure 18: Barplot comparing all four classification methods.

4.4.3 Shapelets

Shapelet Transform learned 10 discriminative subsequences (5 at length=10 weeks, 5 at length=20 weeks) for classification. However, it achieved only 40.08% accuracy with F1-macro of 0.1673, making it the worst-performing method.

Why Shapelets Underperformed:

- Heavy bias toward "Medium" class (recall: 0.99), essentially functioning as a majority-class predictor
- Low mutual information between learned shapelets and rating categories
- Unable to capture the subtle temporal patterns that distinguish rating classes

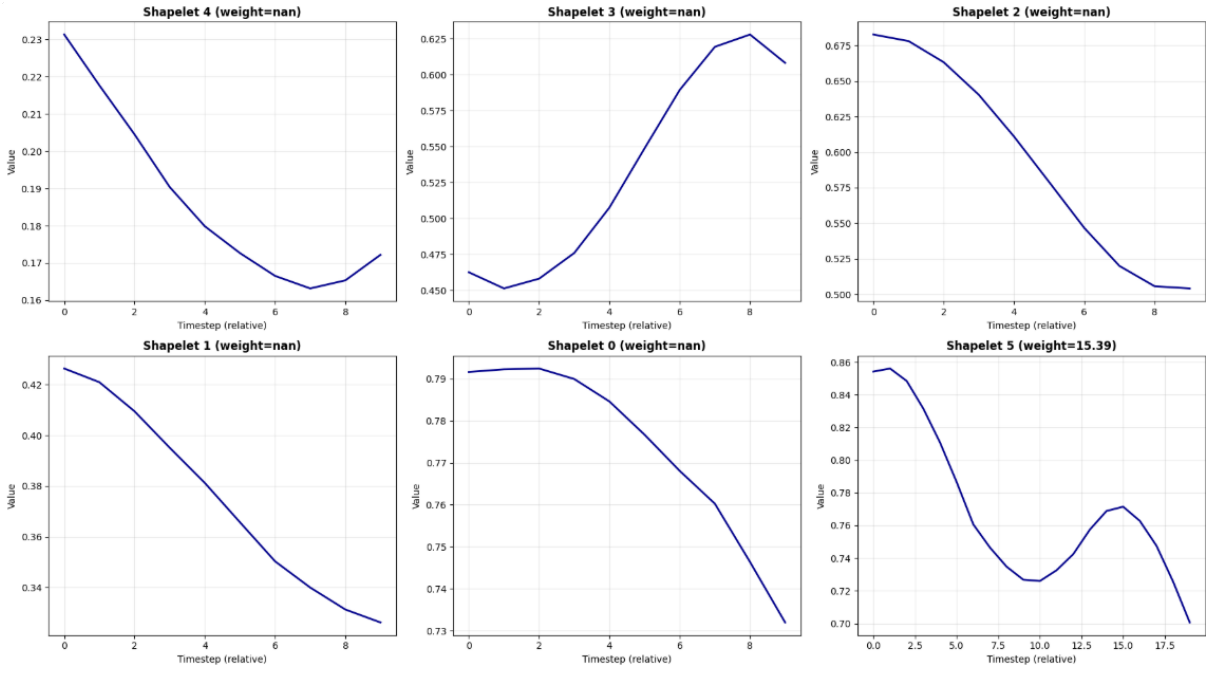

Figure 19: Top-6 learned shapelets showing temporal patterns at lengths 10 and 20 weeks. Despite poor classification, shapelets reveal typical revenue decay shapes.

Shapelets and motifs capture complementary information: shapelets optimize for class discriminability (which failed here), while motifs identify frequently recurring patterns regardless of class labels.

4.5 Time Series Analysis Conclusions

Classification achieved moderate results (40-49% accuracy across methods), with ROCKET performing best (48.90%). Several factors limited performance:

1. **Weak Signal**: Box office revenue patterns show limited correlation with rating categories. Revenue reflects commercial success (marketing budget, star power, release timing), while ratings reflect subjective quality—these are orthogonal dimensions.

2. **Class Imbalance**: With only 10 samples in "Low" class vs 387 in "Medium", algorithms struggled to learn minority patterns, defaulting to majority prediction.

3. **External Factors**: Revenue is influenced by competition (other releases), marketing spend, critical reviews, and cultural trends—none captured in the time series itself.

4. **Dataset Size**: With 1,134 series split 80/20, the test set had only 227 samples across 5 classes (45 per class average), limiting statistical power.

Key Success: Clustering effectively identified blockbusters (7.3% of films with $752M average) vs standard releases, achieving silhouette score of 0.7979. Motifs revealed recurring revenue decay patterns. DTW-based methods consistently outperformed Euclidean distance, validating the importance of temporal alignment for time series comparison.

5 Conclusions and Practical Considerations

5.1 Summary of Results

This project applied advanced data mining techniques to the IMDB dataset (149,521 records, 77 features) for tabular classification and time series analysis (1,134 series). Random Forest achieved best classification performance (44.07% test accuracy), narrowly beating XGBoost (43.95%). Target binning improved F1-macro by 30% (0.3176 to 0.4114). For regression, Random Forest achieved R^2=0.9999 for numVotes prediction (potential overfitting), while XGBoost showed R^2=0.9233 with better generalization. SHAP revealed numerical features (content richness, awards, recency) drive predictions more than genres. Time series classification achieved moderate results with ROCKET performing best (48.90% accuracy), demonstrating that revenue patterns have limited correlation with rating quality.

5.2 Key Findings and Limitations

No model exceeded 44.5% accuracy due to rating subjectivity, class imbalance (minority classes under 1,000 samples), and missing critical features like plot quality. Data leakage detection was critical—initial models achieved over 90% accuracy from popularity_score containing the target. Error analysis revealed: (1) 78% of predictions concentrated in classes 6-8 (majority bias), (2) adjacent class confusion (predicting 7 for 6 or 8), (3) symmetric degradation at extremes (F1 under 0.15 for ratings 1-3, 9-10), and (4) genre-specific blind spots (Documentary+Biography 23% above average error).

SVM trained on only 20% of data due to computational constraints . Time series approximation lost temporal structure. Class imbalance remained unaddressed in main task, affecting minority predictions.